



Звіт про оригінальність

● Оцінка схожості

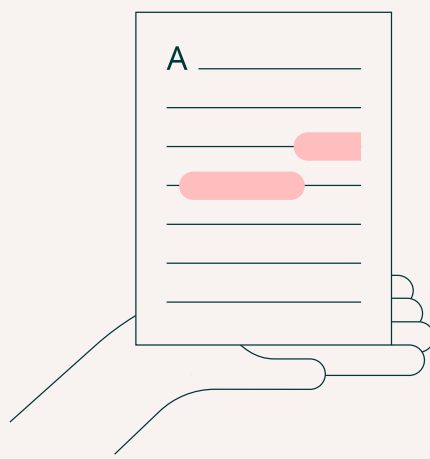
% 6

● Ризик плагіату

СЕРЕДНІЙ

☺ Offiks ⌚ 2026-06-07 13:56

Посилання на звіт: 1ajIF / Посилання користувача: sjWr



Ось вона – Ваша звіт про оригінальність!

Ми раді повідомити, що перевірка вашого документа завершена, і результати вже готові! Наші алгоритми старанно працювали, щоб знайти збіги в наших базах даних.

На наступних сторінках ви знайдете результати перевірки:

Бали

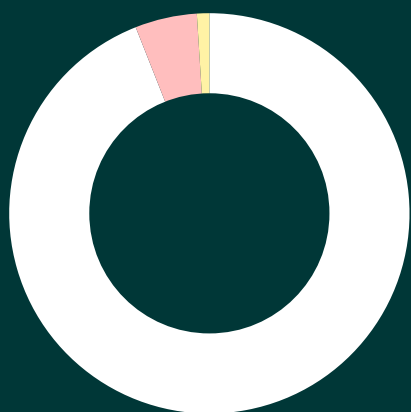
Збіги

Посилання

Ваш документ було перевірено за такими джерелами:

- ☒ База даних інтернет-джерел
- ☐ База даних наукових статей
- ☐ Глибока перевірка (наш вдосконалений алгоритм)

Бали



Збіги тексту	5%
Перефразування	1%
Цитований текст	0%
Неправильне цитування	0%
Збігів не знайдено	94%

Ризик плагіату

СЕРЕДНІЙ

Ризик плагіату вказує, як збіги тексту розподілені по документу. Вищий ризик виникає, коли збіги з'являються близько один до одного, наприклад, у тому самому абзаці або розділі.

Оцінка схожості

Оцінка схожості показує, скільки слів або символів у вашому документі збігаються з текстами інших документів, включаючи перефразовані тексти або неправильні цитати.



6

Збіги

5 МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ ОДЕСЬКИЙ 2 НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ І.І.МЕЧНИКОВА Факультет математики, фізики та інформаційних
технологій Кафедра математичного забезпечення комп'ютерних систем

КУРСОВА РОБОТА з освітнього компоненту «Програмування» на тему: СТВОРЕННЯ
ІЄРАРХІЇ КЛАСІВ «ТЕЛЕФОННИЙ ДОВІДНИК»

Студента 1 курсу, денна форма навчання спеціальність F7 «Комп'ютерна інженерія»

Коробова Олександра Володимировича Керівник: к.ф.- м.н., доц. Антоненко О.С.

5 Національна шкала: _____

5 Кількість балів: _____

ECTS: _____

Дата захисту: _____

Члени комісії ____ Антоненко О.С.

2 (підпис) (прізвище та ініціали)

2 ____ 2 Шестопапов С.В.

2 (підпис) (прізвище та ініціали)

2 ____ 2 _____

2 (підпис) (прізвище та ініціали)

2 ОДЕСА – 2026

ЗМІСТ

стор.

ПОСТАНОВКА ЗАВДАННЯ3

1.ТЕОРЕТИЧНІ ВІДОМОСТІ4

2.СТРУКТУРА ПРОГРАМИ5

2.1Клас «PhoneBook»5

2.2Абстрактний клас «Record»6

2.3Клас «Person»7

2.4Клас «Organization»7

2.5Клас «Phone»8

2.6Опис деяких функцій9

3.ІНТЕРФЕЙС10

ВИСНОВКИ13

ДОДАТОК А14

ПОСТАНОВКА ЗАВДАННЯ

Студент повинен розробити консольний додаток, аналог «Телефонного довідника», мовою C++. Розроблений додаток повинен містити в собі записи (Records) та виконувати такі функції: пошук людини або організації за номером телефону, пошук за електронною поштою; пошук людини за ПІБ, частиною ПІБ; додавання та видалення записів в довіднику; виведення вмісту довідника в консоль. Довідник повинен мати інтерактивний, охайний та інтуїтивно зрозумілий інтерфейс. Проєкт повинен бути написаний з дотриманням принципів ООП, включаючи інкапсуляцію, успадкування, поліморфізм. Проєкт має містити чітку ієрархію класів. Необхідно використовувати вбудовані класи C++, такі як `string`, `vector`, і т. д.

В додатку мають бути реалізовані такі класи:

Довідник — головний клас, містить динамічний масив усіх записів, повинен дозволяти виконувати основні функції, такі як додавання або видалення запису, пошук запису.

Запис у довіднику — батьківський абстрактний клас, містить масив номерів телефонів та масив електронних адрес, використовується як основа під записи. Так як клас є абстрактним, повинен містити віртуальні методи.

Людина — клас-нащадок, повинен містити загальну інформацію про людину, вказівник на організацію (якщо є) та успадковувати номери телефонів та електронні адреси від батьківського класу.

Організація — клас-нащадок, **6** повинен містити загальну інформацію про організацію, **вказівник** на керівника (якщо є), та успадковувати номери телефонів та електронні адреси.

Телефон — окремий клас, містить три категорії (домашній, мобільний та робочий) та три форми (повна, середня та коротка).

ТЕОРЕТИЧНІ ВІДОМОСТІ

Об'єктно-орієнтоване програмування (ООП) — це сукупність методів програмування, що вбачає програму **3** як сукупність об'єктів, що взаємодіють між собою. Воно виникло як протидія ускладненим процесам розробки великого програмного забезпечення, і наразі використовується майже всіма. В ООП кожен об'єкт — це екземпляр класу, а кожен клас є каркасом, який задає **3** стан та поведінку об'єкта. ООП поширює такі принципи:

Інкапсуляція — Приховування внутрішньої реалізації об'єкта та захищення доступу до його даних від випадкових змін. З об'єктом взаємодія повинна відбуватись виключно через публічні методи, які дозволяють виводити або змінювати інформацію в межах класу.

Наслідування — Механізм, який **4** дозволяє створювати нові класи на основі існуючих. **У 4** ньому нащадок успадковує **4** властивості та методи батьківського класу, що робить код більш компактним.

Поліморфізм — Дозволяє об'єктам різних класів з однаковим інтерфейсом виконувати дії по-різному.

Блискучість ООП полягає в спрощенні структури коду до окремих складових, що значно полегшує розуміння, тестування та особливо масштабування коду.

СТРУКТУРА ПРОГРАМИ

Клас «PhoneBook»

Об'єкт цього класу — наш довідник. Він є кінцевою точкою кожного поля даних програми. Уся інформація довідника збирається і сортується тут. Це той клас, з яким користувач працює напяму.

Довідник — набір записів, отже наш клас не містить нічого окрім самих записів. Це

зручно для тих задач, де потрібно проаналізувати весь довідник з початку до кінця, а не його окремі елементи. В PhoneBook будуть реалізовані методи пошуку, додавання та вилучення інформації.

Поля:

`std::vector<Record*> records` — приватне поле, набір посилань на усі записи довідника.

Методи:

`~PhoneBook()` — деструктор, видаляє кожен елемент `records` по черзі.

`void addRecord(Record* record)` — додає в кінець масиву `records` посилання на `record`.

`void removeRecord(int index)` — приймає числове значення, видаляє запис із масиву за цим індексом, обов'язково перевіряє чи існує елемент з цим індексом.

`std::vector<Record*> findByName(const std::string& name) const` — повертає список посилань на записи, в яких виявлено задане ім'я. Метод одразу підходить і для повних ПІБ, і для їх частин.

`std::vector<Record*> findByPhone(const std::string& phoneNumber) const` — повертає масив посилань на записи, в яких виявлено заданий номер телефону. Одночасно перевіряє співпадіння повних, середніх та коротких форм.

`std::vector<Record*> findByEmail(const std::string& searchEmail) const` — повертає масив посилань на записи зі знайденими адресами.

Кожен пошуковий метод використовує гетери нащадків `Records` для порівняння шуканої інформації з приватними полями об'єктів. Пізніше отримання порожніх векторів буде інтерпретовано як відсутність співпадінь.

Абстрактний клас «Record»

Складова нашого довідника. Містить в собі усю інформацію про запис, доповнену класами нащадками, окрім свого розміщення у списку довідника. Слугує як базис для двох типів записів: людей і організацій, адже їх об'єднують однакові ролі та функції, а також наявність номерів телефонів та електронних адрес.

Поля:

`std::vector<Phone> phones` — список номерів телефонів.

`std::vector<std::string> emails` — список email'ів.

Методи:

Конструктор з параметрами.

`virtual ~Record()` — віртуальний деструктор.

`virtual void print() const` — віртуальний метод роздруківки, який буде реалізовуватись в нащадках.

Гетери для масивів `phones` та `emails`.

Клас «Person»

Будучи нащадком `Record`, `Person` виділяється наявністю ПІБ, статі та вказівника на організацію. Так як клас `Organization` поки що не заданий, використаємо попереднє оголошення класу.

Поля:

`std::string name` — ПІБ (або будь-яке ім'я, так як це ні на чого не впливає).

`std::string gender` — стать (не обмежуємо варіантами, так як це також ні на що не впливає).

`Organization* organization` — вказівник на організацію (повинна існувати в довіднику).

Методи:

Конструктор з параметрами.

Гетери `std::string getName() const` та `Organization* getOrganization() const`.

Сетер `void setOrganization(Organization* org)`.

`void print() const` — виводить на екран всю інформацію про людину. Якщо вказівник на організацію нульовий — виводить «немає» замість назви організації. Виводить кожний номер телефону по черзі, вказуючи категорію та усі його формати. Перший номер телефону — головний.

Клас «Organization»

На відміну від `Person`, має назву організації та її вид діяльності, а також вказівник на керівника. Знову використовуємо попереднє оголошення класу.

Поля:

`std::string title` — назва.

`std::string activity` — діяльність.

`Person* manager` — вказівник на керівника (повинен існувати в довіднику).

Методи:

Конструктор з параметрами.

Гетери `std::string getTitle() const` та `Person* getManager() const`.

Сетер `void setManager (Person* mngr)`.

`void print() const` — виводить на екран всю інформацію про організацію. Якщо вказівник на керівника нульовий — виводить «немає». Виводить кожний номер телефону по черзі, вказуючи категорію та усі його формати. Перший номер телефону — головний.

Клас «Phone»

Номер телефону — об'єкт, який має два параметри: категорія та сам номер. Тому зручніше створити клас `Phone` і в ньому задати метод `print()`, який буде охайно виводити інформацію про нього.

Поля:

`std::string category`.

`std::string full_form`.

`std::string medium_form`.

`std::string short_form`.

Методи:

`std::string getCategory() const`.

`std::string getFull() const`.

`std::string getMedium() const`.

`std::string getShort() const`.

Конструктор з параметрами.

`void print() const` — роздруківка.

Усі поля класів знаходяться в приватному доступі. Поля класу `Record` — у захищеному.

Опис деяких функцій

`std::string choosePhoneCategory()` — використовується для того, щоб дати користувачу вибір між трьома категоріями номерів. Її особливість: вибір повинен бути зроблений, інакше програма не дозволить перейти до наступного кроку. Передбачено вибір неіснуючих варіантів, що виходять за межі одиниці та трійки.

`std::vector<Phone> enterPhones()` — дозволяє вводити будь-яку кількість телефонів в запис довідника. Використовується під час додавання запису. Перевіряє введений номер телефону на правильність та одразу переводить заданий номер у його середню та коротку форми. Щоб завершити функцію, або взагалі не додавати номери, користувач повинен ввести порожній рядок.

`std::vector<std::string> enterEmails()` — аналогічно до `enterPhones`, однак формат адреси не обмежується конкретними правилами.

Інтерфейс реалізується за допомогою команди `switch()`. Користувачу висвітлюється меню, яким він керує за допомогою введення цифр.

ІНТЕРФЕЙС

Програма працює в циклі `while(true)`. Щоб вийти з програми, користувач повинен натиснути «0» в головному меню, тоді буде викликано `return 0`.

При запуску програми висвітлюється головне меню (рис 3.1).

1 Рисунок 3.1 - Головне меню

Перший варіант виводить всі записи в довіднику по черзі, позначаючи їх своїм номером (рис. 3.2).

1 Рисунок 3.2 - Фрагмент довідника

Другий варіант дозволяє додати запис в довідник. Спочатку програма питає, чи хоче користувач додати людину, чи організацію. Реалізується також за допомогою `switch()`. «0» викликає `break` та повертає до головного 1 меню (рис 3.3).

1 Рисунок 3.3 - Вибір типу запису

Після вибору, користувач повинен ввести по черзі інформацію про новий запис (рис

3.4). Його буде додано в кінець довідника.

Рисунок 3.4

Третій варіант дозволяє видалити запис під відповідним номером. Користувач знову має змогу ввести «0» щоб повернутися в головне меню.

Четвертий варіант викликає меню пошуку. Користувач повинен обрати метод, за яким буде шукати записи (рис 3.5). Потім програма виведе усі записи, що підійшли під пошук (рис 3.6).

1 Рисунок 3.5 - Вибір критерію пошуку

Рисунок 3.6

ВИСНОВКИ

Створено телефонний довідник, який має змогу виконувати всі необхідні за умовою завдання функції. Реалізовано інтерактивне керування програмою в терміналі та розбиття проєкту на ієрархію класів. Застосовано основні принципи ООП на практиці.

ООП допомагає розбити складне завдання на його прості складові. Такий формат дозволяє чітко бачити кожен компонент програми і за що він відповідає. Ієрархія класів — невід’ємна частина ООП, яка дозволяє створювати зв’язки між об’єктами та даними без плутанини. Наш телефонний довідник — це приклад, як можна купу рядків, типів та задач організовувати в окремі елементи програми і керувати кожним з них від їх задання до перетворення на потрібний результат.

Телефонний довідник чудово демонструє структуру об’єктів та методів в ООП. Кожен довідник — це сукупність записів. Кожен запис розділяється на дві категорії. Кожна категорія має однакове призначення — виведення чи запис інформації, але різні категорії зберігають різну інформацію. Інформація в записах також розбивається на класи, такі як Phone, string, vector, ці класи в свою чергу мають своє призначення і зберігають іншу інформацію. Всі ці класи збираються в один довідник, щоб над ними виконувалися дії. А якщо потрібно змінити щось для окремого класу, чи додати новий клас або поле, інші не потрібно підлаштовувати під зміни.

ДОДАТОК А

Діаграма класів

Рисунок А.1 — діаграма класів

Посилання

Це джерела виділених збігів у вашому документі. Кожен збіг позначено темно-зеленим числом, яке відповідає вказаному тут джерелу. Джерела впорядковані за схожістю — чим вищий бал, тим сильніше збіг.

#	Джерело	%
1	core.ac.uk	1.4%
2	dspace.onu.edu.ua	1.4%
3	essuir.sumdu.edu.ua	0.8%
4	reporter.zp.ua	0.8%
5	essuir.sumdu.edu.ua	0.6%
6	tras.mali.cx.ua	0.5%



Дякуємо, що перевірили
свій документ за допомогою
Plag!